

Space-Invaders

Definiere jedes Interface in einer eigenen .java-Datei, die den Namen des Interfaces trägt, z. B. `IBasicGameObject.java` für `IBasicGameObject`.

1 IBasicGameObject

Aufgabe: Lege das Interface `IBasicGameObject` an und deklariere die Methoden

- `List<StringWithLocation> show()`
- `List<V2> hitBox()`
- `boolean isAlive(List<IBasicGameObject> gameObjects, int width, int height)`

2 BasicGameObject

2.1 BasicGameObject — isAlive

Aufgabe: Ergänze `boolean isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `BasicGameObject`. Die Methode soll immer `true` zurückgeben.

2.2 Aufgabe

Ergänze in der Klasse `BasicGameObject` die Interface-Deklaration und `@Override`-Annotationen.

2.3 Aufgabe

Ergänze die Methode `boolean checkCollision(IBasicGameObject other)` in `BasicGameObject`. Sie soll `true` zurückgeben, wenn die Hitbox dieses Objekts mit der Hitbox von `other` kollidiert.

```
// zusätzliche Beispiele für Kollisionstests
var c1 = new BasicGameObject(new V2(2,2), "XY");
var c2 = new BasicGameObject(new V2(3,2), "A");
var c3 = new BasicGameObject(new V2(4,2), "B");
println(c1.checkCollision(c2));
println(c1.checkCollision(c3));
```

```
true
false
```

Hinweis: Nutze `Utils.intersect`

2.4 Aufgabe

Ergänze die Methode `boolean checkCollision(List<IBasicGameObject> gameObjects)` in `BasicGameObject`. Sie prüft ob eine Kollision mit einem anderen Objekt aus `gameObjects` vorliegt. Die übergebene Liste enthält immer das Objekt selbst.

```
var b1 = new BasicGameObject(new V2(3,4), "xy");
var b2 = new BasicGameObject(new V2(3,4), "#");
var b3 = new BasicGameObject(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.checkCollision(allGameObjects));
println(b1.checkCollision(List.of(b1)));
```

```
true
false
```

Hinweis: Nutze `checkCollision`

2.5 Aufgabe

Ergänze die korrekte Implementierung von `isAlive` in `BasicGameObject`.

Erklärung: `isAlive(...)` prüft, ob das Objekt noch Teil des Spiels ist. Es soll `true` liefern wenn

- mindestens eine Zelle der `hitBox()` auf dem Spielfeld ist, und
- keine Kollision mit einem anderen Objekt aus `gameObjects` vorliegt (gemeinsame V2 in Hitboxes). Die Liste kann dieses Objekt selbst enthalten; dann gilt `collisionCount > 1` als Hinweis auf Kollision mit einem anderen Objekt.

```
var b1 = new BasicGameObject(new V2(3,4), "xy");
var b2 = new BasicGameObject(new V2(3,4), "#");
var b3 = new BasicGameObject(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 3, 4));
```

```
false
true
false
```

Hinweis: Nutze `Utils.isOnBoard` und die `checkCollision`-Methoden von `BasicGameObject`.

3 MovableGameObject

3.1 Aufgabe

Ergänze die Methode `boolean isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `MovableGameObject`. Die Logik soll die gleiche sein wie in `BasicGameObject`, aber die Methode soll die Implementierung von `BasicGameObject` wiederverwenden.

```
var b1 = new MovableGameObject(new V2(3,4), "xy");
var b2 = new BasicGameObject(new V2(3,4), "#");
var b3 = new BasicGameObject(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

3.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `MovableGameObject`.

```
var b1 = new MovableGameObject(new V2(3,4), "xy");
var b2 = new MovableGameObject(new V2(3,4), "#");
var b3 = new BasicGameObject(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

4 Alien

4.1 Aufgabe

Ergänze die Methode boolean `isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `Alien`. Verwende dabei die Methode der Klasse `BasicGameObject`.

```
var b1 = new Alien(new V2(3,4), "xy");
var b2 = new MovableGameObject(new V2(3,4), "#");
var b3 = new BasicGameObject(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

4.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `Alien`.

5 AlienRocket

5.1 Aufgabe

Ergänze die Methode boolean `isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `AlienRocket`. Verwende dabei die Methode der Klasse `BasicGameObject`.

```
var b1 = new AlienRocket(new V2(3,4));
var b2 = new MovableGameObject(new V2(3,4), "#");
var b3 = new Alien(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

5.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `AlienRocket`.

6 Player

6.1 Aufgabe

Ergänze die Methode boolean `isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `Player`. Verwende dabei die Methode der Klasse `BasicGameObject`.

```
var b1 = new Player(new V2(3,4));
var b2 = new AlienRocket(new V2(3,4));
var b3 = new Alien(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

6.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `Player`.

7 PlayerRocket

7.1 Aufgabe

Ergänze die Methode `boolean isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `PlayerRocket`. Verwende dabei die Methode der Klasse `BasicGameObject`.

```
var b2 = new PlayerRocket(new V2(3,4));
var b1 = new Player(new V2(3,4));
var b3 = new AlienRocket(new V2(10,10));
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
false
true
false
```

7.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `PlayerRocket`.

8 Plasma

8.1 Aufgabe

Ergänze die Methode `boolean isAlive(List<IBasicGameObject> gameObjects, int width, int height)` in `Plasma`. Verwende dabei die Methode der Klasse `BasicGameObject`.

```
var b1 = new Plasma(new V2(5,5), 3);
var b2 = new PlayerRocket(new V2(3,4));
var b3 = new Alien(new V2(10,10), "#");
List<IBasicGameObject> allGameObjects = List.of(b1, b2, b3);
println(b1.isAlive(allGameObjects, 100, 60));
println(b1.isAlive(List.of(b1), 100, 60));
println(b1.isAlive(List.of(b1), 4, 4));
```

```
true
true
false
```

8.2 Aufgabe

Ergänze die korrekte Interface-Deklaration und `@Override`-Annotationen für das Interface `IBasicGameObject` in der Klasse `Plasma`.

9 Utils

9.1 Aufgabe

Ergänze die statische Methode `List<StringWithLocation> getStringsWithLocation(List<IBasicGameObject> basicGameObjects)` in der Klasse `Utils`. Die Methode soll die `show()`-Listen aller übergebenen `IBasicGameObject` zusammenfügen und als eine Liste zurückgeben.

```
var b1 = new BasicGameObject(new V2(3,4), "X");
var m1 = new MovableGameObject(new V2(5,5), "Y");
List<IBasicGameObject> allGameObjects = List.of(b1, m1);
println(Utils.getStringsWithLocation(allGameObjects).size());
```

```
2
```

9.2 Aufgabe

Ergänze die statische Methode `<T extends IBasicGameObject> List<T> removeDeadObjects(List<T> gameObjectsToFilter, List<IBasicGameObject> allGameObjects, int width, int height)` in der Klasse `Utils`. Die Methode soll alle Objekte aus `gameObjectsToFilter` herausfiltern, die laut `isAlive(allGameObjects, width, height)` nicht mehr leben, und die verbleibenden Objekte zurückgeben.

```
var a1 = new Alien(new V2(3,4), "A");
var a2 = new Alien(new V2(3,4), "A");
List<IBasicGameObject> allGameObjects = List.of(a1, a2);
List<Alien> aliensToFilter = List.of(a1, a2);
println(Utils.removeDeadObjects(aliensToFilter, allGameObjects, 100, 60).size());
```

10 Rocket (Aufgaben)

10.1 Aufgabe

Lege das Interface `Rocket` an. Das Interface soll `IBasicGameObject` erweitern und die folgenden Methoden deklarieren:

- `Rocket move()`
- `boolean isPlayerRocket()`

10.2 Aufgabe

Ergänze bei den Klassen, die Raketen repräsentieren (`PlayerRocket`, `AlienRocket`, `Plasma`) die Interface-Deklaration `implements Rocket` und setze `@Override` über `move()` und `isPlayerRocket()`.

```
var pr = new PlayerRocket(new V2(10,5));
var ar = new AlienRocket(new V2(5,5));
var pl = new Plasma(new V2(5,5), 3);
List<IBasicGameObject> allGameObjects = List.of(pr, ar, pl);
println(pr.isPlayerRocket());
println(ar.isPlayerRocket());
println(pl.isPlayerRocket());
```

```
true
false
true
```

11 Utils

11.1 Aufgabe

Ergänze die statische Methode `List<Rocket> move(List<Rocket> rockets)` in der Klasse `Utils`. Die Methode soll jede Rakete in der übergebenen Liste bewegen (durch Aufruf von `move()` auf jedem `Rocket`) und die resultierende Liste zurückgeben.

```
var r1 = new PlayerRocket(new V2(10,10));
var r2 = new AlienRocket(new V2(5,5));
List<Rocket> rockets = List.of(r1, r2);
var moved = Utils.move(rockets);
println(moved);
```

```
[PlayerRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=9], displayString=|
^]]], AlienRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=6], displayString=|
^]]]]
```

12 containsNoPlayerRocket

12.1 Aufgabe

Ergänze die statische Methode `boolean containsNoPlayerRocket(List<Rocket> rockets)` in der Klasse `Utils`. Die Methode soll `true` zurückgeben, wenn in der Liste keine `Rocket` existiert, bei der `isPlayerRocket()` `true` ist.

```
var r1 = new PlayerRocket(new V2(10,10));
var r2 = new AlienRocket(new V2(5,5));
List<Rocket> rockets1 = List.of(r2);
println(Utils.containsNoPlayerRocket(rockets1)); // true
List<Rocket> rockets2 = List.of(r1, r2);
println(Utils.containsNoPlayerRocket(rockets2)); // false
```

```
true
false
```

13 Model

14 Aufgabe

Ergänze in der Klasse `Model` die Eigenschaft `rockets` als `List<Rocket>`. Die Methode `restart` von `Model` soll eine leere Liste von Raketen initialisieren.

```
var model = new Model(10, 20);  
model.restart();  
println(model.rockets);
```

```
[]
```

15 Aufgabe

Ergänze in der Klasse Model die Methode moveRockets(), die alle Raketen in rockets bewegt und die Liste durch die neue Liste ersetzt.